

SMAART INSTITUTE

Hosting & Deployment Guide

A simple, complete record of how the student site and the admin site were put live on AWS, how new versions are deployed, and what to do day to day.

Deployment done on 22 August 2026 · Document prepared 23 August 2026

Item	Value
Student site	https://smaartinstitute.com
Admin site	https://admin.smaartinstitute.com
Server (Elastic IP)	13.207.12.89
AWS region	ap-south-1 (Mumbai)
Instance type	t3.medium (2 vCPU, 4 GB RAM), 40 GB gp3 disk
Operating system	Ubuntu 26.04 LTS
How the apps run	Docker – 6 containers, one edge nginx in front
Database	MongoDB Atlas (cloud, shared by both apps)
HTTPS certificate	Let's Encrypt – renews automatically
Deployment	GitHub Actions – build automatically, deploy with one click

Contents

1. What we have now (summary)	4
1.1 Two websites, one server	4
1.2 What was achieved.....	4
2. The problem we had to solve	5
2.1 The fix: an edge proxy	5
3. How the server is arranged	6
3.1 The files that make it work	6
3.2 Two things fixed while building this	6
3.3 A trap worth knowing: two different URL styles	7
4. Code fixes made before going live	8
4.1 Two wider findings (still open)	8
5. Step by step: how the hosting was done.....	9
Step 1 – Launch the EC2 instance (AWS Console)	9
Step 2 – Attach an Elastic IP	9
Step 3 – Connect and install Docker (on the server)	9
Step 4 – Deploy keys and cloning the repos (on the server)	10
Step 5 – Copy the two .env files (from the laptop)	11
Step 6 – DNS records (Hostinger)	11
Step 7 – Get the HTTPS certificate (on the server).....	11
Step 8 – First deployment (on the server).....	11
Step 9 – Fix the uploads permission error.....	12
Step 10 – Verify everything	12
6. How deployment works now (CI/CD)	13
6.1 Why this design	13
6.2 The workflows	13
6.3 Safety built into the deploy	14
6.4 How to deploy (the click).....	14
6.5 GitHub secrets used.....	14
7. Day-to-day operations (cheat sheet).....	15
Connect to the server (from the laptop)	15
Everyday checks (on the server).....	15
Health check (from anywhere).....	15
Deploy a new version	15
Emergency – GitHub is unavailable	15
After editing a .env file	15

Service names.....	16
8. Every error we hit, and how it was solved	17
Error 1 – ^[[200~ corrupting every paste	17
Error 2 – Pastes silently not registering.....	17
Error 3 – clip failing without a word	17
Error 4 – Two different .pem files	17
Error 5 – chmod 600 not taking on Windows	18
Error 6 – Running laptop commands on the server (repeatedly)	18
Error 7 – ssh: no key found, then [none publickey]	18
Error 8 – Secrets that were never secret.....	18
Error 9 – Rollback that would have made things worse.....	18
Error 10 – ghcr.io/v2/: denied	18
Error 11 – "Run workflow" appearing to do nothing	19
Error 12 – A secret name that would not save.....	19
9. What is still outstanding	20
9.1 How to rotate (short version).....	20
10. The 10 most important questions (and answers)	21
Appendix A – Simple glossary.....	23
Appendix B – Important values and paths	24

If the list above is empty, right-click it in Word and choose "Update Field".

1. What we have now (summary)

In one day the SMAART Institute applications went from an empty AWS launch screen to two live websites on HTTPS, with a safe one-click deployment process. This document explains, in simple words, what was done and how to use it.

1.1 Two websites, one server

	Student dashboard	Admin panel
Website	smaartinstitute.com	admin.smaartinstitute.com
GitHub repo	SMAART-INSTITUTE-USERDASHBOARD	ADMIN-SMAART-INSTITUTE-26
Frontend	Vite + React	Create React App
Backend	Express on port 5000	Express on port 5001
Used by	Students	Staff / admins
Database	MongoDB Atlas (same cluster)	MongoDB Atlas (same cluster)

1.2 What was achieved

- Both sites are live on HTTPS (the padlock icon). HTTP automatically redirects to HTTPS.
- One server runs both apps using Docker. Only one container (the "edge") is open to the internet.
- The HTTPS certificate renews itself every 90 days – no manual work.
- The database is connected and email / OTP login works.
- Every push to GitHub builds the apps automatically. Deploying to the live site is a deliberate button click by a person.
- If a deployment fails its health check, the server rolls back to the previous version by itself.
- Five security / correctness bugs in the code were found and fixed before going live.

2. The problem we had to solve

The requirement was clear: the student dashboard and the admin panel must both run on the same EC2 server, as two separate websites.

Each project came with its own docker-compose.yml, and each one assumed it owned the whole machine. The admin panel wanted to publish its frontend on port 80 (the normal web port). The student app wanted port 8081 for its frontend and port 80 through its own nginx. Only one program can hold port 80 on a machine – so if you start both, the second one simply fails.



Figure 1 – Before and after. One edge proxy owns the public ports and forwards each request to the right app.

2.1 The fix: an edge proxy

We added one small nginx container called the edge. It is the only thing that listens on ports 80 and 443. It reads the domain name from each request and passes it to the correct app:

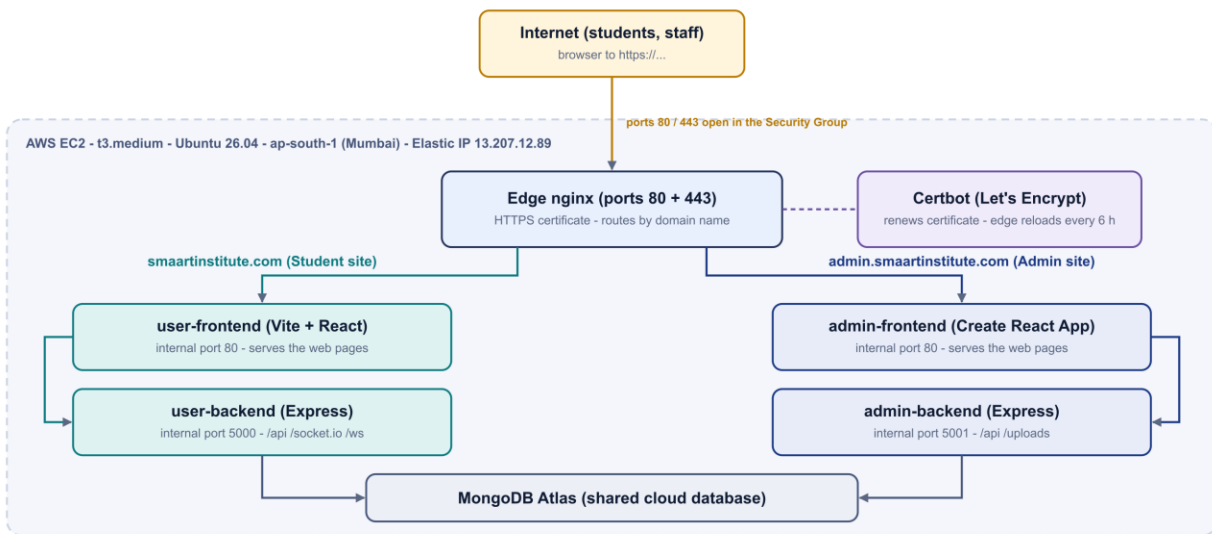
- **smaartinstitute.com** → student frontend; /api, /socket.io, /ws → student backend (port 5000)
- **admin.smaartinstitute.com** → admin frontend; /api, /uploads → admin backend (port 5001)

All the application containers became internal only. Nobody on the internet can reach ports 5000 or 5001 directly.

3. How the server is arranged

How the server is arranged

One EC2 machine (13.207.12.89) - Docker - Only the edge container is open to the internet



Six containers in total: edge, certbot, user-frontend, user-backend, admin-frontend, admin-backend. Ports 5000 and 5001 are never opened to the internet.

Figure 2 – Six Docker containers on one EC2 instance. The edge handles HTTPS and routing; certbot keeps the certificate fresh.

3.1 The files that make it work

Five files were added to the dashboard repo at `aws-deployment/single-instance/`:

File	What it does
<code>docker-compose.prod.yml</code>	Describes all six containers. Only the edge publishes ports to the host.
<code>edge.conf</code>	nginx configuration: HTTPS (TLS) and routing by domain name.
<code>bootstrap.sh</code>	One-time script that gets the Let's Encrypt certificate.
<code>deploy.env.example</code>	Template for the domains and the Let's Encrypt email.
<code>README.md</code>	Full server runbook for the person operating the server.

3.2 Two things fixed while building this

- **Certificate renewal was silently useless.** Certbot writes a new certificate every 60–90 days, but a running nginx keeps using the old one it loaded at start. Around day 90 the site would have shown an expired-certificate error. Fix: the edge now reloads its configuration every 6 hours.
- **Certbot was on its own throwaway network.** It was moved onto the shared Docker network so nginx and certbot can talk to each other.

3.3 A trap worth knowing: two different URL styles

Variable	App	Includes /api?
VITE_API_URL	Student	Yes – e.g. https://smaartinstitute.com/api
REACT_APP_API_URL	Admin	No – bare origin, the code adds /api itself
REACT_APP_STUDENT_API_URL	Admin	No – bare origin

This is not a bug to "normalise". The compose file already sets each one correctly. Also: the admin app calls the student API directly from the browser, so the student backend's CORS allow-list must contain both domains, or admin pages that read student data will fail.

4. Code fixes made before going live

Before touching AWS, the code was read carefully. Five problems would have caused real trouble in production. All were fixed and syntax-checked with `node --check` before committing.

#	Problem found	Why it matters	Fix applied
1	Login OTP codes were printed to the log file on every login (back-end/utis/emailService.js).	Anyone who could read logs/combined.log could log in as any student.	Logging now happens only when <code>NODE_ENV</code> is development or <code>LOG_OTP=true</code> .
2	A master secret (<code>ADMIN_SYSTEM_SECRET</code>) was hard-coded in an admin frontend file and also sat in Git history.	One import away from being shipped to every browser.	Hard-coded secret removed; the secret was rotated to a new 64-character random value.
3	A developer "bypass" header was still honoured in production by <code>requireRole</code> (roleMiddleware.js).	Any logged-in student sending the header + secret became an admin – including the power to delete students.	Bypass now only works when <code>NODE_ENV</code> is not production. Confirmed nothing legitimate used it.
4	Vision Board code built the URL as <code>/api/api/ocr/extract</code> (imageModeration.js).	Every Vision Board upload would return 404 in production.	Double <code>/api</code> corrected in three places.
5	An old note said Gmail SMTP was broken (535 BadCredentials).	Would block OTP login emails.	Stale note – the account had already been changed to <code>smaart.sys@gmail.com</code> with a working App Password. Proven with <code>nodemailer.verify()</code> inside the live container.

4.1 Two wider findings (still open)

Four credentials are still visible in Git history. `back-end/.env` is ignored by Git today, but it was committed in three earlier commits (49e0ca57, 921850b6, fe8aa068). Removing a file from the repo does not remove it from history.

Secret	Still the leaked value?
MONGODB_URI	Yes
JWT_SECRET	Yes – the serious one: it signs login tokens
CLOUDINARY_API_SECRET	Yes
OPENROUTER_API_KEY	Yes
ADMIN_SYSTEM_SECRET	No – rotated
Gmail password	No – replaced

A third service that does not exist. Both backends have `ITSM_URL=http://localhost:5002`. Inside a container, "localhost" means that container itself, and there is no ITSM service in either repo. Tickets still save (the bridge is non-blocking) – they just never reach ITSM.

5. Step by step: how the hosting was done

This is the exact order the work was done in. Each step says where to run it (laptop or server) and how to know it worked.

Which machine am I on?

Laptop: dhars@LAPTOP-KB5MB70S MINGW64 – Git commits, scp, anything with a C:\ or /c/ path

Server: ubuntu@ip-172-31-13-117:~\$ – anything with docker compose

Step 1 – Launch the EC2 instance (AWS Console)

Setting	Value	Why
AMI	Ubuntu 26.04 LTS	Supported until 2031; Docker publishes packages for it (verified on download.docker.com, not assumed).
Instance type	t3.medium	4 GB RAM. A t3.micro cannot finish a Create React App build.
Storage	40 GB gp3	218 MB of face-api models + two node_modules folders + uploaded images.
Security group	Ports 22, 80, 443 only	Never open 5000 or 5001 – they stay behind the proxy.
Key pair	smaart-prod.pem	Saved to ~/.ssh on the laptop.

Caught at the launch screen

"Allow HTTP traffic" was unticked. Port 80 is not only for visitors – Let's Encrypt uses it to prove you own the domain. With it closed, the certificate script fails and no certificate is ever issued.

Step 2 – Attach an Elastic IP

An Elastic IP was allocated and attached, fixing the address at 13.207.12.89. Without it the IP changes every time the instance stops and starts, which breaks DNS and the MongoDB Atlas allow-list.

Step 3 – Connect and install Docker (on the server)

Docker Desktop is a laptop product. Servers use Docker Engine – no account, no licence. Run these four commands one at a time (one long paste was corrupted – see Section 8, error 1).

```
# connect from the laptop
ssh -i /c/Users/dhars/.ssh/smart-prod.pem ubuntu@13.207.12.89

# 1 - prerequisites and Docker's signing key
sudo apt-get update && sudo apt-get install -y ca-certificates curl git && sudo install -m 0755 -
d /etc/apt/keyrings && sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc && sudo chmod a+r /etc/apt/keyrings/docker.asc

# 2 - add the Docker repository (detects "resolute" = 26.04 by itself)
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" |
sudo tee /etc/apt/sources.list.d/docker.list

# 3 - install Docker Engine + Compose
sudo apt-get update && sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin

# 4 - permissions + 4 GB swap (needed for the frontend builds)
sudo usermod -aG docker $USER && sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile &&
sudo mkswap /swapfile && sudo swapon /swapfile && echo '/swapfile none swap sw 0 0' | sudo tee -a
/etc/fstab
```

Result: Docker 29.7.2, Compose 5.5.0, 4 GB swap. **Log out and reconnect after step 4** – the docker group only applies on a fresh login.

Step 4 – Deploy keys and cloning the repos (on the server)

Both repos are private, so the server needs its own read-only access. We used deploy keys – one per repo, scoped to that repo only, and revocable. A personal token would have given access to the whole GitHub account.

```
# one key per repo
ssh-keygen -t ed25519 -C "ec2-smaart-dashboard" -f ~/.ssh/dashboard_key -N ""
ssh-keygen -t ed25519 -C "ec2-smaart-admin" -f ~/.ssh/admin_key -N ""

# tell SSH which key belongs to which repo
printf 'Host github-dashboard\n  HostName github.com\n  User git\n  IdentityFile
~/.ssh/dashboard_key\n  IdentitiesOnly yes\n\nHost github-admin\n  HostName github.com\n  User
git\n  IdentityFile ~/.ssh/admin_key\n  IdentitiesOnly yes\n' > ~/.ssh/config && chmod 600
~/.ssh/config

# show the public keys -> add each under GitHub > repo > Settings > Deploy keys (read-only)
cat ~/.ssh/dashboard_key.pub; cat ~/.ssh/admin_key.pub

# verify - GitHub replies with the repo name each key belongs to
ssh -T git@github-dashboard; ssh -T git@github-admin

# clone into /opt/smaart
sudo mkdir -p /opt/smaart && sudo chown $USER:$USER /opt/smaart && cd /opt/smaart
git clone git@github-dashboard:Abdur-Rahim-MyGit/SMAART-INSTITUTE-USERDASHBOARD.git SMAART-
INSTITUTE-USERDASHBOARD
git clone git@github-admin:Abdur-Rahim-MyGit/ADMIN-SMAART-INSTITUTE-26.git SMAART-
INSTITUTE-ADMIN
```

The folder name is not optional

The admin repo is called ADMIN-SMAART-INSTITUTE-26 on GitHub, but it must land in a folder called SMAART-INSTITUTE-ADMIN, because that is the path the compose file builds from. Leave out the folder name and the build fails with a confusing "path not found".

Step 5 – Copy the two .env files (from the laptop)

The .env files hold the secrets and are ignored by Git, so they were copied by hand with scp:

```
scp -i /c/Users/dhars/.ssh/smaart-prod.pem "/c/Users/dhars/Desktop/SMAART-INSTITUTE/SMAART-
INSTITUTE-USERDASHBOARD/back-end/.env" ubuntu@13.207.12.89:/opt/smaart/SMAART-INSTITUTE-
USERDASHBOARD/back-end/.env
scp -i /c/Users/dhars/.ssh/smaart-prod.pem "/c/Users/dhars/Desktop/SMAART-INSTITUTE-
ADMIN/Backend/.env" ubuntu@13.207.12.89:/opt/smaart/SMAART-INSTITUTE-ADMIN/Backend/.env
```

Step 6 – DNS records (Hostinger)

The domain is smaartinstitute.com (not smaartminds.com, which appears in older guides in the repo). Two A records point both names at the server:

Type	Name	Value	Meaning
A	@	13.207.12.89	the domain itself – smaartinstitute.com
A	admin	13.207.12.89	admin.smaartinstitute.com

At first only the admin record existed. A query to Google DNS confirmed it was not a propagation delay – the answer had no A record at all. The same query showed the nameservers (ns1.dns-parking.com / dns.hostinger.com), which told us the record had to be added in Hostinger.

```
curl -s "https://dns.google/resolve?name=smaartinstitute.com&type=A"
curl -s "https://dns.google/resolve?name=smaartinstitute.com&type=NS"
```

Step 7 – Get the HTTPS certificate (on the server)

```
cd /opt/smaart/SMAART-INSTITUTE-USERDASHBOARD/aws-deployment/single-instance
cp deploy.env.example deploy.env      # then edit the domains + email if needed
chmod +x bootstrap.sh && sed -i 's/\r$//' bootstrap.sh && ./bootstrap.sh
```

The certificate was issued in about twenty seconds. One certificate covers both hostnames, lives under /etc/letsencrypt/live/smaartinstitute.com/, expires 20 November 2026 and renews automatically. The sed removes Windows line endings – cheap insurance against a baffling \$'\r': command not found error.

Step 8 – First deployment (on the server)

```
nohup docker compose -f docker-compose.prod.yml up -d --build > /tmp/build.log 2>&1 &
tail -f /tmp/build.log
```

It is run detached (nohup ... &) because if the terminal disconnects mid-build, the build would otherwise be killed.

Step 9 – Fix the uploads permission error

Five of six containers started. The student backend crash-looped with:

```
Error: EACCES: permission denied, mkdir '/usr/src/app/uploads/proctoring'
```

Reason: back-end/uploads/ is ignored by Git, so a fresh clone does not have it. When a bind-mount folder is missing, Docker creates it as root. The student backend deliberately runs as the non-root "node" user, so it could not write there. (The admin backend was fine only because its Dockerfile runs as root – which is why this was confusing.)

```
CU=$(docker run --rm smaart-prod-user-backend id -u):$(docker run --rm smaart-prod-user-backend id -g)
sudo chown -R $CU /opt/smaart/SMAART-INSTITUTE-USERDASHBOARD/back-end/uploads
docker compose -f docker-compose.prod.yml restart user-backend
```

Step 10 – Verify everything

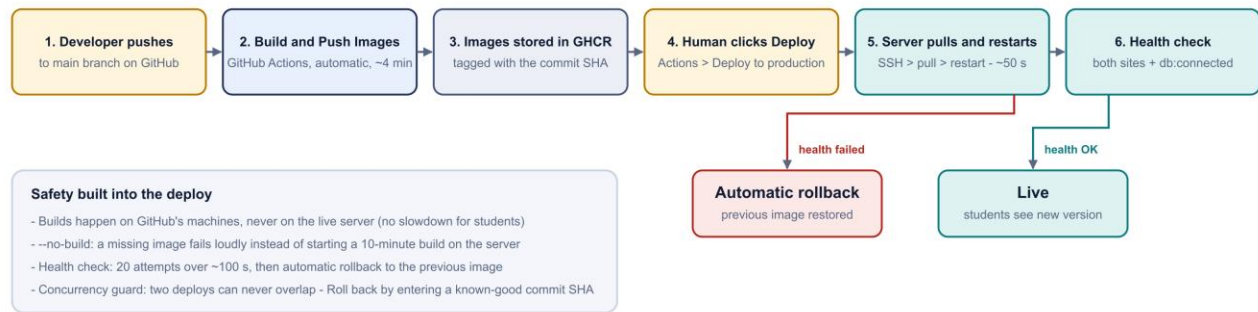
```
curl -sI https://smaartinstitute.com | head -1      # HTTP/2 200
curl -s https://smaartinstitute.com/api/health     # {"status":"Server is
running","db":"connected"}
curl -sI https://admin.smaartinstitute.com | head -1  # HTTP/2 200
curl -sI http://smaartinstitute.com | head -1      # HTTP/1.1 301 (redirects to https)
```

All six containers up, both sites returning 200, database connected, HTTP redirecting to HTTPS. The sites were live.

6. How deployment works now (CI/CD)

How a new version reaches the live site

Pushing code never deploys by itself. A person must click "Deploy to production".



The Deploy button lives only in the dashboard repo, because one docker-compose.prod.yml there runs both apps.

Figure 3 – Pushing code builds images automatically. Putting them live is a deliberate human click.

6.1 Why this design

Two options were weighed. Building on the server was rejected: the frontend build needed 4 GB of swap on an idle machine, so running it while students are on the site would make the site crawl and risk running out of memory. Instead, GitHub's machines build the Docker images and store them in GHCR (GitHub Container Registry). The server only pulls ready-made images and restarts – about 50 seconds.

Pushing never deploys. That is deliberate for a live student system.

6.2 The workflows

Workflow	Repo	Trigger	What it does
Build & Push Images	both	push to main + manual	Builds Docker images on GitHub, pushes to GHCR tagged with the commit SHA (~4 min).
Deploy to production	dashboard only	manual only	SSH to server → pull images → restart → health check → rollback on failure.
CI / CI-CD Pipeline	both	push	Kept – they run the tests.
Legacy EC2 deploys	both	disabled	Old scripts that did <code>rm -rf /var/www/html</code> , <code>pm2 restart</code> etc. Switched to manual-only with an explanation, not deleted.

The Deploy button lives only in the dashboard repo, because the single docker-compose.prod.yml there runs both apps. Two buttons could collide mid-restart.

6.3 Safety built into the deploy

- `--no-build` – if an image is missing, the run fails loudly instead of silently starting a ten-minute build on the live server.
- **Health check** – 20 attempts over about 100 seconds, testing both sites and `"db":"connected"`.
- **Automatic rollback** – if the health check fails, the previous image is restored.
- **Concurrency guard** – two deploys can never overlap.
- **Immutable rollback reference** – the deploy records the image digest (or local image ID), not the moving `"latest"` tag, so a rollback really goes back.
- **Local fallback** – `docker compose up --build` still works on the server, so a GitHub outage is not a lock-out.

6.4 How to deploy (the click)

1. Push or merge your code into the main branch of the dashboard repo (and/or the admin repo).
2. Wait for "Build & Push Images" to go green in the Actions tab (about 4 minutes).
3. Dashboard repo → Actions → "Deploy to production" → click the grey "Run workflow ✓" button.
4. Leave both image tags as "latest" → click the green "Run workflow" button.
5. Watch the run. Within about 50 seconds it shows the health check passing and the site is updated.

To roll back on purpose: run the same workflow, but enter the commit SHA of a known-good build instead of `"latest"`.

6.5 GitHub secrets used

Secret	Purpose
EC2_SSH_KEY	Private key GitHub uses to SSH into the server.
GHCR_PULL_TOKEN	Token the server uses to pull images from GHCR.

EC2_HOST and EC2_USER are hard-coded in the workflow, not secrets – the IP is public in DNS and `"ubuntu"` is the stock user. A stray space in a secret gives the same error as a broken key, and secrets cannot be read back to check.

7. Day-to-day operations (cheat sheet)

Connect to the server (from the laptop)

```
ssh -i /c/Users/dhars/.ssh/smaart-prod.pem ubuntu@13.207.12.89
```

Everyday checks (on the server)

```
cd /opt/smaart/SMAART-INSTITUTE-USERDASHBOARD/aws-deployment/single-instance

docker compose -f docker-compose.prod.yml ps          # all six should read Up
docker compose -f docker-compose.prod.yml logs -f user-backend # follow one service (Ctrl+C to stop)
docker compose -f docker-compose.prod.yml restart user-backend # restart one service
```

Health check (from anywhere)

```
curl -sI https://smaartinstitute.com | head -1
curl -s https://smaartinstitute.com/api/health
curl -sI https://admin.smaartinstitute.com | head -1
```

Deploy a new version

Dashboard repo → Actions → Deploy to production → grey "Run workflow ✓" → leave both as latest → green "Run workflow". (Full steps in Section 6.4.)

Emergency – GitHub is unavailable

```
cd /opt/smaart/SMAART-INSTITUTE-USERDASHBOARD && git pull
cd /opt/smaart/SMAART-INSTITUTE-ADMIN && git pull
cd /opt/smaart/SMAART-INSTITUTE-USERDASHBOARD/aws-deployment/single-instance
docker compose -f docker-compose.prod.yml up -d --build
```

After editing a .env file

```
docker compose -f docker-compose.prod.yml up -d --force-recreate
```

Important

restart is NOT enough after changing a .env – it reuses the old environment. Use --force-recreate.

Service names

Service	What it is	Internal port
edge	nginx – HTTPS + routing (the only public one)	80, 443 (published)
certbot	Let's Encrypt renewal	–
user-frontend	Student website pages	80
user-backend	Student API (Express)	5000
admin-frontend	Admin website pages	80
admin-backend	Admin API (Express)	5001

8. Every error we hit, and how it was solved

This is the most valuable section. Most of these will happen again to whoever does this next.

Error 1 – ^[[200~ corrupting every paste

What happened: Terminals send a "bracketed paste" marker that this shell did not understand. It corrupted the first line of every pasted block, which broke quoting and turned the rest into nonsense (getent: command not found, #: command not found). Commands appeared to run, so it silently wasted two rounds of debugging.

Fix:

```
bind 'set enable-bracketed-paste off'
echo "bind 'set enable-bracketed-paste off'" >> ~/.bashrc
```

Note: Type the first line by hand once; the second makes it permanent.

Error 2 – Pastes silently not registering

What happened: The single biggest time cost of the day. Both the SSH key and the GHCR token "looked wrong" when in fact the field was empty – the paste never landed. Hours went into replacing perfectly good credentials.

Fix:

```
read -rsp "paste token: " T && echo && echo "characters received: ${#T}"
```

Note: "characters received: 0" means the paste failed. 40 means the token is real and the problem is elsewhere. Always prove a credential arrived before deciding it is bad.

Error 3 – clip failing without a word

What happened: cat ~/.ssh/gh_deploy | clip in Git Bash gave no error and no clipboard content.

Fix:

```
Get-Content "$env:USERPROFILE\.ssh\gh_deploy" -Raw | Set-Clipboard
```

Note: Use PowerShell, or write to a file and copy from an editor.

Error 4 – Two different .pem files

What happened: Downloads held smart-key.pem (4 July) and Smaart-key (1).pem (today). Different keys – the older one belonged to a previous instance. Using it gives "Permission denied (publickey)".

Fix:

```
md5sum "smart-key.pem" "Smaart-key (1).pem"
cp "$USERPROFILE/Downloads/Smaart-key (1).pem" ~/.ssh/smaart-prod.pem
```

Note: Compare by hash and date, then install the right one under a clean name.

Error 5 – chmod 600 not taking on Windows

What happened: Git Bash reports success but Windows uses ACLs, so SSH still saw a world-readable key.

Fix:

```
icacls "$env:USERPROFILE\.ssh\smaart-prod.pem" /inheritance:r /grant:r "$($env:USERNAME):(R)"
```

Note: Run in PowerShell.

Error 6 – Running laptop commands on the server (repeatedly)

What happened: `cd: /c/Users/dhars/...: No such file or directory` – that path exists only on Windows.

Fix: Rule: `C:\` or `/c/` paths and any git commit belong on the laptop. Anything with docker compose belongs on the server. Check the prompt before typing.

Error 7 – ssh: no key found, then [none publickey]

What happened: Two different failures that look alike. The first means the key would not parse. The second means it parsed and was offered, but the server rejected it.

Fix: The decisive test was using the same key pair from the laptop – it worked. That proved the key was fine and moved the search to `EC2_USER` and `EC2_HOST`.

Error 8 – Secrets that were never secret

What happened: `EC2_HOST` and `EC2_USER` were stored as GitHub secrets. Neither is secret, but a stray space in either gives the identical error as a broken SSH key, and secrets cannot be read back to check.

Fix: Hard-coded both in the workflow. Only `EC2_SSH_KEY` and `GHCR_PULL_TOKEN` remain as secrets. This fixed the SSH failure immediately.

Error 9 – Rollback that would have made things worse

What happened: The deploy recorded the tag of the running container. Deploying "latest" recorded "latest" – a moving pointer. A rollback would have reinstalled the same broken image while reporting success.

Fix: Record an immutable reference instead: the registry digest, or the local image ID when the image was built locally. Two sub-bugs: `RepoDigests` lives on the image, not the container; and locally-built images have no digest, so a fallback to image ID was needed.

Error 10 – ghcr.io/v2/: denied

What happened: Traced with error 2's technique to an empty `GHCR_PULL_TOKEN` secret. The token itself was valid – "Login Succeeded" when tested directly on the server.

Fix: Re-paste the secret and confirm its length.

Error 11 – "Run workflow" appearing to do nothing

What happened: There are two buttons with that name.

Fix: The grey one opens a panel; the green one inside it actually starts the run.

Error 12 – A secret name that would not save

What happened: "Secret names can only contain alphanumeric characters or underscores." Caused by hyphens or a trailing space from pasting the name.

Fix: Type GHCR_PULL_TOKEN by hand.

9. What is still outstanding

Item	Status
Both sites live on HTTPS	Done
Certificate auto-renewing	Done
Database connected	Done
Email / OTP working	Done
CI building both apps	Done
One-click deploy, proven	Done
Legacy workflows disabled	Done
Runbooks in both repos	Done
Immutable rollback	Pushed, not yet exercised – watch the next deploy log for "previous user-backend : sha256:..."
Delete Desktop\EC2_SSH_KEY.txt on the laptop	To do
Rotate the four leaked credentials	To do – the one thing that still blocks real students

Why rotating the secrets matters

MONGODB_URI, JWT_SECRET, CLOUDINARY_API_SECRET and OPENROUTER_API_KEY are readable in Git history and are still the live values on the server.

JWT_SECRET signs login tokens: anyone holding it can forge an admin session with no password. The repos are private, which limits exposure to whoever has repo access – but private is not the same as rotated.

Rotating now costs about twenty minutes and nothing else. Rotating after launch logs every student out mid-session.

9.1 How to rotate (short version)

1. Generate new values locally: `openssl rand -hex 48` for JWT_SECRET.
2. MongoDB Atlas → Database Access → edit the user → new password → update MONGODB_URI.
3. Cloudinary → Settings → Access Keys → generate a new pair, disable the old one.
4. openrouter.ai → Keys → create a new key, delete the old one, set a credit limit.
5. Update both .env files on the server (JWT_SECRET and MONGODB_URI must be identical in both).
6. `docker compose -f docker-compose.prod.yml up -d --force-recreate`, then run the health checks.

10. The 10 most important questions (and answers)

These are the questions most likely to be asked in a review, a hand-over or an interview about this deployment. Each answer is short enough to say out loud.

Q1. Why did you need an edge proxy? Could you not just run both apps?

No. Both apps wanted port 80, and only one program can hold a port. The edge nginx is the only thing listening on 80/443; it reads the domain name in each request and forwards it to the right app. The apps themselves are never exposed to the internet.

Q2. Why Docker on the server instead of pm2 + nginx installed directly?

Each app ships with a Dockerfile, so Docker gives the same build on the laptop, on GitHub and on the server. Six containers on one compose file are easy to start, stop and roll back as a unit, and nothing leaks between the two apps.

Q3. Why does pushing to main NOT deploy automatically?

It is a live student system. Pushing builds the images (safe, on GitHub's machines). A person then clicks "Deploy to production". This is the gate that the two old legacy workflows were missing – they deployed on every push with `rm -rf` and `pm2 restart`.

Q4. Why build on GitHub and not on the server?

The frontend build needed 4 GB of swap on an idle machine. Running it while students are online would slow the site and risk out-of-memory. The server only pulls finished images and restarts – about 50 seconds.

Q5. What happens if a deployment is broken?

The workflow runs a health check (20 tries over ~100 s) on both sites and the database. If it fails, it automatically rolls back to the previous image. The rollback uses an immutable digest / image ID, not the "latest" tag, so it truly goes back.

Q6. How does HTTPS work and how is it renewed?

Let's Encrypt issued one free certificate covering both domains via `bootstrap.sh` (port 80 must be open for the ownership check). A `certbot` container renews it, and the edge nginx reloads every 6 hours so it actually picks up the new file.

Q7. Why an Elastic IP?

Without it, the server's IP changes every stop/start, which breaks DNS (both A records) and the MongoDB Atlas IP allow-list. The Elastic IP pins it to 13.207.12.89.

Q8. Why deploy keys instead of a personal access token?

Both repos are private. A deploy key is read-only, scoped to one repository, and can be revoked without touching anything else. A personal token would give the server access to the whole GitHub account.

Q9. What was the most serious code issue found before go-live?

A privilege-escalation bypass: `requireRole` still honoured a developer header in production. Any logged-in student sending the header plus a secret that was hard-coded in a frontend file could become an admin – including deleting students. It is now disabled in production. OTP codes were also being written to log files.

Q10. What is still risky, and what should be done next?

Four credentials (MongoDB URI, JWT secret, Cloudinary secret, OpenRouter key) are visible in Git history and still live on the server. `JWT_SECRET` alone lets someone forge an admin session. They must be rotated – about twenty minutes now, versus logging out every student if done after launch.

Appendix A – Simple glossary

Word	Simple meaning
EC2	A virtual computer rented from AWS. Ours is in Mumbai.
Elastic IP	A fixed public IP address that stays with our server.
Security group	The firewall for the EC2 instance. We allow only ports 22, 80, 443.
Docker / container	A way to package an app with everything it needs so it runs the same everywhere. Each running package is a container.
docker compose	A file + command that starts several containers together.
Image	The built, ready-to-run package of an app. Containers are started from images.
GHCR	GitHub Container Registry – where our built images are stored.
nginx / edge / reverse proxy	A web server that sits in front, handles HTTPS and forwards requests to the right app.
Port	A numbered "door" on a machine. 80 = HTTP, 443 = HTTPS, 22 = SSH.
DNS / A record	The phone book of the internet. An A record maps a name (smaartinstitute.com) to an IP address.
Let's Encrypt / certbot	A free service that issues HTTPS certificates; certbot is the tool that asks for and renews them.
SSH	Secure remote terminal into the server, using a key file (.pem).
Deploy key	A read-only SSH key that gives one server access to one GitHub repo.
.env file	A text file of secrets and settings (database URL, passwords). Never committed to Git.
GitHub Actions / workflow	Automation that runs on GitHub when code is pushed or a button is clicked.
CI/CD	Continuous Integration / Continuous Delivery – automatically test, build and deliver code.
Health check	A quick request to the site to confirm it is up and the database is connected.
Rollback	Going back to the previous working version.
Commit SHA	The unique ID of a Git commit; our images are tagged with it.
Swap	Disk space used as extra memory when RAM runs out.
CORS	A browser rule that decides which websites may call an API.

Appendix B – Important values and paths

Item	Value
Student site	https://smaartinstitute.com
Admin site	https://admin.smaartinstitute.com
Health endpoint	https://smaartinstitute.com/api/health
Server IP (Elastic IP)	13.207.12.89
Server user	ubuntu
SSH key on the laptop	/c/Users/dhars/.ssh/smaart-prod.pem
Code on the server	/opt/smaart/SMAART-INSTITUE-USERDASHBOARD and /opt/smaart/SMAART-INSTITUTE-ADMIN
Compose folder	/opt/smaart/SMAART-INSTITUE-USERDASHBOARD/aws-deployment/single-instance
Compose file	docker-compose.prod.yml
Certificate	/etc/letsencrypt/live/smaartinstitute.com/ (expires 20 Nov 2026, auto-renews)
DNS provider	Hostinger (ns1.dns-parking.com)
Database	MongoDB Atlas – shared by both apps
Email account	smaart.sys@gmail.com (App Password)
Full server runbook	aws-deployment/single-instance/README.md
Day-to-day deploy guide	DEPLOY.md